Actividad: Punteros

Programación I, Semana 11

Nombres: _____ y _____

Sección: _____

Esta actividad **no tiene nota**.

Ejercicios obligatorios

Ejercicio 1: Tabla de memoria

Suponer que `x` está en la dirección **1000**, `y` en la **1004** y `p` en la **1008**. Completar la tabla con el valor de cada variable **después** de ejecutar cada línea. Si la variable todavía no existe, escribir un guion (`-`).

Pista: en la columna `p` va una dirección. `*p` no es una variable: es «lo que hay donde apunta `p`».

```
int x = 5;      // 1
int y = 8;      // 2
int* p = &x;    // 3
*p = 7;         // 4
p = &y;         // 5
*p = *p + x;    // 6
```

Línea	x	y	p
1	5	-	-
2			
3			
4			
5			
6			

Al final, ¿qué imprime `std::cout << *p;`? _____

Ejercicio 2: ¿Qué imprime?

a)

```
int a = 3;
int* p = &a;
int* q = p;
*q = 10;
std::cout << a << " " << *p;
```

Imprime: _____

b)

```
int v[4] = {2, 4, 6, 8};
int* p = v;
p[1] = 9;
std::cout << v[1] << " " << p[3];
```

Imprime: _____

c)

Pista: observar cuál de los dos parámetros es un puntero.

```
void f(int x, int* y) {
    x = x * 2;
    *y = *y * 2;
}

int main() {
    int a = 4;
    int b = 4;
    f(a, &b);
    std::cout << a << " " << b;
    return 0;
}
```

Imprime: _____

Ejercicio 3: Intercambiar

Completar la función para que intercambie los valores de las dos variables, y completar la llamada en `main`.

Pista: hace falta una variable auxiliar. Dentro de la función, el valor de cada variable se obtiene con `*`.

```
void intercambiar(int* a, int* b) {
    int temp = __________;
    __________ = __________;
    __________ = temp;
}

int main() {
    int x = 3;
    int y = 9;
    intercambiar(__________, __________);
    std::cout << x << " " << y;
    return 0;
}
```

Resultado esperado: el programa imprime **9 3**.

Ejercicio 4: Encontrar los errores

Este programa debería aplicar un descuento del 50 % a un lápiz e imprimir su precio y el doble de una cantidad, pero tiene **4 errores**. Encerrar cada uno en un círculo y escribir al lado la corrección.
Resultado esperado: ya corregido, el programa imprime **1 6**.

```
struct Producto {
    std::string nombre;
    double precio;
};

void aplicarDescuento(Producto* p, double porcentaje) {
    p.precio = p.precio - p.precio * porcentaje / 100;
}

int main() {
    Producto lapiz = {"Lapiz", 2.0};
    aplicarDescuento(lapiz, 50);

    int cantidad = 3;
    int* c = cantidad;
    int* total;
    *total = *c * 2;

    std::cout << lapiz.precio << " " << *total;
    return 0;
}
```

Ejercicios intermedios

Ejercicio 5: Cuenta bancaria

```
struct Cuenta {
    std::string titular;
    double saldo;
};
```

Escribir **completas** estas dos funciones, usando `->`:

- `void depositar(Cuenta* c, double monto)`: suma el monto al saldo.
- `bool retirar(Cuenta* c, double monto)`: si el saldo alcanza, resta el monto y devuelve `true`; si no, no cambia nada y devuelve `false`.

Resultado esperado: con `Cuenta cuenta = {"Ana", 100};`, después de `depositar(&cuenta, 50)` el saldo es **150**. Luego `retirar(&cuenta, 200)` devuelve **false** y el saldo sigue en 150; `retirar(&cuenta, 30)` devuelve **true** y el saldo queda en **120**.

Reto

Ejercicio 6: Buscar una cuenta

```
const int N = 4;
Cuenta banco[N] = {{"Ana", 100}, {"Luis", 250}, {"Maria", 80}, {"Pedro", 40}};
```

a) Escribir la función `Cuenta* buscarCuenta(Cuenta c[], int n, std::string titular)`, que devuelve un **puntero** a la cuenta de ese titular, o `nullptr` si no existe.

Pista: la dirección del elemento `i` es `&c[i]`.

b) Escribir el código de `main` que busca la cuenta de Maria y, si existe, le deposita 20 con la función `depositar` del ejercicio 5.

Resultado esperado: el saldo de `banco[2]` pasa a **100**. Con `"Sofia"`, la función devuelve `nullptr` y no se deposita nada.